Python Assignment – Functions, Iterators and Generators

Student Name

Nirmalya Das

Subject

Python Programming

Assignment Title

Python Functions, Iterators, Generators and Lambda Expressions

---

1. What is the difference between a function and a method in Python?

Answer:

A **function** is a block of reusable code that performs a specific task and can be called independently. It is defined using the `def` keyword.

A **method** is a function that is associated with an object or class and is called using that object.

Example of a Function

```
def greet():
  print("Hello World")

greet()
```

Example of a Method

```
text = "hello"
print(text.upper())
```

In this example, `upper()` is a **method** because it belongs to the string object `text`.

---

2. Explain the concept of function arguments and parameters in Python.

Answer:

In Python functions, **parameters** are variables defined in the function declaration, while **arguments** are the actual values passed to the function when it is called.

Example

```
def add(a, b):   # a and b are parameters
  return a + b

result = add(5, 3)   # 5 and 3 are arguments
print(result)
```

Here:

- a and b are parameters
- 5 and 3 are arguments

Parameters receive the values passed as arguments during function execution.

---

3. What are the different ways to define and call a function in Python?

Answer:

Python allows functions to be defined and called in several ways.

1. Simple Function

```
def greet():
  print("Hello")

greet()
```

2. Function with Parameters

```
def add(a, b):
  print(a + b)
```

```
add(4, 6)
```

3. Function with Return Value

```
def multiply(a, b):
  return a * b

print(multiply(3, 5))
```

These variations allow functions to handle different programming needs.

---

4. What is the purpose of the `return` statement in a Python function?

Answer:

The `return` statement is used to **send a result back from a function to the place where it was called**.

It allows functions to produce output that can be used later in the program.

Example

```
def square(num):
  return num * num

result = square(4)
print(result)
```

Output:

```
16
```

The function returns the square of the number instead of just printing it.

---

5. What are iterators in Python and how do they differ from iterables?

Answer:

An **iterable** is an object that can be looped over using a loop such as `for`.

Examples include:

- lists
- tuples
- sets
- dictionaries
- strings

An **iterator** is an object that keeps track of the current position during iteration and implements the methods `__iter__()` and `__next__()`.

Example

```
numbers = [1, 2, 3]

iterator = iter(numbers)

print(next(iterator))
print(next(iterator))
```

Here, `numbers` is an **iterable**, while `iterator` is an **iterator** created using `iter()`.

---

6. Explain the concept of generators in Python and how they are defined.

Answer:

A **generator** is a special type of function that produces values **one at a time instead of returning all values at once**.

Generators are defined using the `yield` keyword instead of `return`.

Example

```
def count_up_to(n):
  i = 1
  while i <= n:
    yield i
```

```
    i += 1

for number in count_up_to(3):
  print(number)
```

Output:

```
1
2
3
```

Generators are useful when working with large datasets.

---

7. What are the advantages of using generators over regular functions?

Answer:

Generators offer several advantages:

1. **Memory Efficient** – They generate values one at a time instead of storing them all in memory.
2. **Faster Execution** – Values are produced only when needed.
3. **Simpler Code** – Generators simplify iteration logic.

Example

```
def generator_example():
  yield 1
  yield 2
  yield 3

for value in generator_example():
  print(value)
```

This approach is more efficient than creating a list containing all values at once.

---

8. What is a lambda function in Python and when is it typically used?

Answer:

A **lambda function** is a small anonymous function defined using the `lambda` keyword.

It is typically used for short, simple operations that do not require a full function definition.

Example

```
square = lambda x: x * x
print(square(5))
```

Output:

25

Lambda functions are commonly used with functions like `map()`, `filter()`, and `sorted()`.

---

9. Explain the purpose and usage of the `map()` function in Python.

Answer:

The `map()` function applies a given function to each element of an iterable and returns a new iterable containing the results.

Example

```
numbers = [1, 2, 3, 4]

result = map(lambda x: x * 2, numbers)

print(list(result))
```

Output:

[2, 4, 6, 8]

Here, the lambda function multiplies each number in the list by 2.

10. What is the difference between `map()`, `reduce()`, and `filter()` functions in Python?

Answer:

These functions are used for functional programming in Python.

Function	Purpose
map()	Applies a function to each element in an iterable
filter()	Selects elements that satisfy a condition
reduce()	Applies a function cumulatively to reduce the iterable to a single value

Example of map()

```
numbers = [1,2,3]
result = map(lambda x: x*2, numbers)
print(list(result))
```

Example of filter()

```
numbers = [1,2,3,4]

result = filter(lambda x: x % 2 == 0, numbers)

print(list(result))
```

Example of reduce()

```
from functools import reduce

numbers = [1,2,3,4]

result = reduce(lambda a,b: a+b, numbers)

print(result)
```

Output:

```
10
```

These functions are widely used for data processing and transformation.

11. Using pen & Paper write the internal mechanism for sum operation using reduce function on this given list:[47,11,42,13]

(11) Internal Mechanism for sum Operation using reduce function on this given list: [47,11,4213]

→ The 'reduce()' function from the 'functools' module applies a function cumulatively to the elements of an iterable, reducing the iterable to a single final value.

For a sum operation, the function repeatedly adds two elements at a time.

Steps-

Step 1 → 47 + 11 → 58

Step 2 → 58 + 42 → 100

Step 3 → 100 + 13 → 113

Result - 113

Python Code -

```
from functools import reduce
number = [47, 11, 42, 13]
result = reduce(lambda x, y: x+y, numbers)
print (result)
```

Output -

113

Thus, reduce() functions iteratively combines elements of the list until one final value remains.